



Thought for 1m 54s >

了解です。

「**Global Growth Console**（世界レベルの意思決定コンソール）」を実装するための、**Claude**向け詳細仕様 + コード例 + **Wolfram Language**スクリプトをまとめます。

そのままコピペして渡せるように書きます。

CQOx Global Growth Console 実装仕様（Claude向け）

0. ゴールと前提

ゴール

- ページ名（仮）：`Decision Console - Marketing Decisions v2`
- 目的:
 - 直近 14 / 28 / 56 日のマーケティング施策ポートフォリオを
 - インクリメンタル利益（ $\Delta\text{¥}$ ）
 - リスク（CVaR や分散）
 - 因果品質スコア（CAS）を軸に“一枚で意思決定できる”ダッシュボードとして可視化する。
- 対象ユーザ:
 - CMO / マーケティング責任者: 週次レビュー用
 - 施策オーナー / データサイエンティスト: どの施策を深掘りするか決める入口

非ゴール（やらないこと）

- MMM（Media Mix Modeling）の実装そのもの
- オフラインの最適化（すでに Portfolio ページが担当）
- クリエイティブ単位の詳細（将来拡張）

1. データモデル & メトリクス定義

1.1 既存テーブルの想定

※正確なスキーマはリポジトリに合わせて調整してください。ここでは「論理モデル」を定義します。

analysis_runs

- `id: UUID` — 因果分析のランID（Causal Design で1回「TRAIN MODELS」した単位）
- `dataset_id: str`
- `scenario_name: str`
- `started_at: datetime`
- `completed_at: datetime`
- `status: enum("completed", "failed", ...)`

policy_evaluations

- 1行 = 1つの「ポリシー（施策）」 × 1分析ラン
- 主キー: (`analysis_run_id`, `policy_id`)
- 列:
 - `analysis_run_id: UUID`
 - `policy_id: str`
 - `policy_name: str`
 - `channel: str | null` 例: "email", "app_push", "multi-channel"
 - `segment_label: str | null` 例: "High LTV", "RFM 4-5-5"
 - `delta_yen: float` — 推定インクリメンタル利益
 - `roi: float` — $\Delta\text{¥} / \text{cost}$

- `risk: float` — CVaR 10% または標準偏差
- `cas: float` — Causal Assurance Score (0~1)
- `verdict: enum("go", "canary", "hold")`
- `period_start: date`
- `period_end: date`
- `budget: float` — 推定/実コスト
- `reach_users: int | null` — 推定リーチ人数

segments

- セグメントメタ
 - `segment_id: str`
 - `name: str`
 - `incremental_clv: float` — 1ユーザあたりΔLTV
 - `user_count: int`
 - `rfm_r: int`
 - `rfm_f: int`
 - `rfm_m: int`

※ RFM はオプション。なければ null でよい。

1.2 メトリクス定義 (Python/Wolfram共通)

1.2.1 期間フィルタ

`from <= period_end <= to` でフィルタされた `policy_evaluations` の集合を `S` とする。

1.2.2 サマリーKPI

- **Total Incremental Profit (Δ¥)**

$$\Delta Y_{\text{total}} = \sum_{s \in S} s.\text{delta_yen}$$

- **Average Δ¥ per Policy**

$$\Delta Y_{\text{avg}} = \frac{1}{|S|} \sum_{s \in S} s.\text{delta_yen}$$

- **Mean CAS**

$$\text{CAS}_{\text{mean}} = \frac{1}{|S|} \sum_{s \in S} s.\text{cas}$$

- **Portfolio CVaR (worst 10%)**

全ポリシーの利得分布から下位10%平均。実装簡便化のため Δ¥ の分布で代用してもよい。

Python 擬似コード:

```
import numpy as np

def compute_cvar(delta_values: list[float], alpha: float = 0.1) -> float:
    if not delta_values:
        return 0.0
    sorted_vals = np.sort(delta_values)
    cutoff_index = max(1, int(len(sorted_vals) * alpha))
    worst_tail = sorted_vals[:cutoff_index]
    return float(np.mean(worst_tail))
```

Wolfram Language 例:

```
deltaVals = policyData[All, "delta_yen"];
alpha = 0.1;
sorted = Sort[deltaVals];
cutoff = Max[1, Floor[Length[sorted]*alpha]];
cvar = Mean[Take[sorted, cutoff]];
```

1.2.3 Segment Opportunity Map 指標

各 `segment_id` について：

- `incremental_clv` (1ユーザーあたりΔLTV)
- `user_count`
- `segment_delta_yen = Sum[delta_yen]` (当該セグメントをターゲットにした施策のΔ¥合計)
- `segment_cas_mean`
- これを横軸/縦軸/バブルサイズ/色に対応付ける。

BCG風ラベルは

- High CLV threshold: 上位 30% 分位
- Large Size threshold: 上位 30% 分位
など分位で決める。

2. バックエンド実装仕様 (FastAPI 想定)

2.1 新規API一覧

1. `GET /api/v1/console/global-summary`
2. `GET /api/v1/console/delta-trend`
3. `GET /api/v1/console/segment-opportunity`
4. `GET /api/v1/console/channel-performance`
5. `GET /api/v1/console/decision-cards`

共通クエリパラメータ:

- `from: date` (必須)
- `to: date` (必須)
- `channel: str | None`
- `segment: str | None`

2.2 Pydantic モデル例

```
# backend/engine/models/console.py

from datetime import date
from pydantic import BaseModel
from typing import List, Optional

class GlobalSummary(BaseModel):
    total_delta_yen: float
    avg_delta_yen_per_policy: float
    mean_cas: float
    cvar_worst10: float
    budget_used: float
    budget_plan: Optional[float] = None

class DeltaTrendPoint(BaseModel):
    week_label: str # e.g. "2025-W48"
    delta_yen: float
    go_delta_yen: float
    canary_delta_yen: float
    hold_delta_yen: float

class SegmentBubble(BaseModel):
    segment_id: str
    name: str
    incremental_clv: float
    user_count: int
    total_delta_yen: float
    mean_cas: float
    bcg_quadrant: str # "Star" | "Volume" | "Niche" | "Low"
```

```

class ChannelPoint(BaseModel):
    channel: str
    total_delta_yen: float
    risk: float
    mean_cas: float

class DecisionCard(BaseModel):
    policy_id: str
    policy_name: str
    dataset: str
    channel: Optional[str]
    segment_label: Optional[str]
    delta_yen: float
    roi: float
    cas: float
    risk: float
    verdict: str # "go" | "canary" | "hold"
    period_start: date
    period_end: date

```

2.3 FastAPI ルーター一例

```

# backend/gateway/routers/console.py

from fastapi import APIRouter, Depends, Query
from datetime import date
from typing import List, Optional

from backend.engine.services.console import (
    get_global_summary,
    get_delta_trend,
    get_segment_opportunity,
    get_channel_performance,
    get_decision_cards,
)
from backend.engine.models.console import (
    GlobalSummary,
    DeltaTrendPoint,
    SegmentBubble,
    ChannelPoint,
    DecisionCard,
)

router = APIRouter(prefix="/api/v1/console", tags=["console"])

@router.get("/global-summary", response_model=GlobalSummary)
async def api_global_summary(
    from_date: date = Query(..., alias="from"),
    to_date: date = Query(..., alias="to"),
    channel: Optional[str] = None,
    segment: Optional[str] = None,
):
    return await get_global_summary(from_date, to_date, channel, segment)

@router.get("/delta-trend", response_model=List[DeltaTrendPoint])
async def api_delta_trend(
    from_date: date = Query(..., alias="from"),
    to_date: date = Query(..., alias="to"),
):
    return await get_delta_trend(from_date, to_date)

@router.get("/segment-opportunity", response_model=List[SegmentBubble])
async def api_segment_opportunity(
    from_date: date = Query(..., alias="from"),

```

```

    to_date: date = Query(..., alias="to"),
):
    return await get_segment_opportunity(from_date, to_date)

@router.get("/channel-performance", response_model=List[ChannelPoint])
async def api_channel_performance(
    from_date: date = Query(..., alias="from"),
    to_date: date = Query(..., alias="to"),
):
    return await get_channel_performance(from_date, to_date)

@router.get("/decision-cards", response_model=List[DecisionCard])
async def api_decision_cards(
    from_date: date = Query(..., alias="from"),
    to_date: date = Query(..., alias="to"),
    verdict: Optional[str] = None,
):
    return await get_decision_cards(from_date, to_date, verdict)

```

2.4 集計ロジック (サービス層)

```

# backend/engine/services/console.py

from datetime import date
from typing import Optional, List
from sqlalchemy import select, func

from .db import SessionLocal
from .models.console import (
    GlobalSummary, DeltaTrendPoint, SegmentBubble, ChannelPoint, DecisionCard
)
from .schema import PolicyEvaluation, Segment # ORM models
from .metrics import compute_cvar # 前述の関数

async def get_global_summary(from_date: date, to_date: date,
                             channel: Optional[str], segment: Optional[str]) -> GlobalSummary:
    with SessionLocal() as db:
        q = db.query(PolicyEvaluation).filter(
            PolicyEvaluation.period_end >= from_date,
            PolicyEvaluation.period_end <= to_date,
        )
        if channel:
            q = q.filter(PolicyEvaluation.channel == channel)
        if segment:
            q = q.filter(PolicyEvaluation.segment_label == segment)

        rows = q.all()
        deltas = [r.delta_yen for r in rows]
        cas_vals = [r.cas for r in rows]
        total_delta = float(sum(deltas)) if deltas else 0.0
        avg_delta = float(total_delta / len(deltas)) if deltas else 0.0
        mean_cas = float(sum(cas_vals) / len(cas_vals)) if cas_vals else 0.0
        cvar = compute_cvar(deltas, alpha=0.1) if deltas else 0.0
        budget_used = float(sum(r.budget or 0.0 for r in rows))

    return GlobalSummary(
        total_delta_yen=total_delta,
        avg_delta_yen_per_policy=avg_delta,
        mean_cas=mean_cas,
        cvar_worst10=cvar,
        budget_used=budget_used,
        budget_plan=None, # TODO: 予算テーブルと連携するならここで
    )

```

他の `get_delta_trend` / `get_segment_opportunity` / `get_channel_performance` / `get_decision_cards` も同様に、ORM から集計して `Pydantic` モデルで返す。

2.5 バックエンド実行・テストコマンド例

```
# バックエンド起動
cd backend
uvicorn gateway.app:app --reload

# 単体テスト (新規テストファイル追加想定)
pytest tests/test_console_endpoints.py -q
```

3. フロントエンド実装仕様 (React + Vite + TanStack Query)

3.1 ルーティング

- 既存の `Decision Console` ページと同一パス (例: `/console`)。
- 新UIは **サマリー-KPI + 3チャート + Decision Cards テーブル** の構成。

3.2 TypeScript 型

```
// frontend/src/types/console.ts

export interface GlobalSummary {
  total_delta_yen: number;
  avg_delta_yen_per_policy: number;
  mean_cas: number;
  cvar_worst10: number;
  budget_used: number;
  budget_plan?: number | null;
}

export interface DeltaTrendPoint {
  week_label: string;
  delta_yen: number;
  go_delta_yen: number;
  canary_delta_yen: number;
  hold_delta_yen: number;
}

export interface SegmentBubble {
  segment_id: string;
  name: string;
  incremental_clv: number;
  user_count: number;
  total_delta_yen: number;
  mean_cas: number;
  bcg_quadrant: "Star" | "Volume" | "Niche" | "Low";
}

export interface ChannelPoint {
  channel: string;
  total_delta_yen: number;
  risk: number;
  mean_cas: number;
}

export interface DecisionCard {
  policy_id: string;
  policy_name: string;
  dataset: string;
  channel: string | null;
  segment_label: string | null;
  delta_yen: number;
  roi: number;
  cas: number;
}
```

```

    risk: number;
    verdict: "go" | "canary" | "hold";
    period_start: string; // ISO
    period_end: string;   // ISO
  }

```

3.3 クライアントAPI クエリ

```

// frontend/src/lib/consoleClient.ts

import { client } from "../client"; // 既存のfetchラッパ
import {
  GlobalSummary,
  DeltaTrendPoint,
  SegmentBubble,
  ChannelPoint,
  DecisionCard,
} from "@types/console";

export async function fetchGlobalSummary(params: {
  from: string;
  to: string;
}): Promise<GlobalSummary> {
  return client.get("/api/v1/console/global-summary", { params });
}

export async function fetchDeltaTrend(params: {
  from: string;
  to: string;
}): Promise<DeltaTrendPoint[]> {
  return client.get("/api/v1/console/delta-trend", { params });
}

// 他の fetchSegmentOpportunity, fetchChannelPerformance, fetchDecisionCards も同様

```

3.4 ページコンポーネント骨格

```

// frontend/src/pages/DecisionConsolePage.tsx

import { useState } from "react";
import { useQuery } from "@tanstack/react-query";
import {
  fetchGlobalSummary,
  fetchDeltaTrend,
  fetchSegmentOpportunity,
  fetchChannelPerformance,
  fetchDecisionCards,
} from "@lib/consoleClient";
import {
  GlobalSummary,
  DeltaTrendPoint,
  SegmentBubble,
  ChannelPoint,
  DecisionCard,
} from "@types/console";

export function DecisionConsolePage() {
  const [range, setRange] = useState<"14d" | "28d" | "56d">("28d");

  const { from, to } = resolveRange(range); // 日付Rangeを返すヘルパ

  const summaryQuery = useQuery<GlobalSummary>({
    queryKey: ["console-summary", from, to],
    queryFn: () => fetchGlobalSummary({ from, to }),
  });

  const trendQuery = useQuery<DeltaTrendPoint[]>({

```

```

    queryKey: ["console-trend", from, to],
    queryFn: () => fetchDeltaTrend({ from, to }),
  });

  // segmentQuery, channelQuery, cardsQuery も同様

  return (
    <div className="page decision-console">
      { /* 期間切り替えボタン */ }
      <HeaderWithRangeSelector range={range} onChange={setRange} />

      { /* KPI カード */ }
      <KPICards summary={summaryQuery.data} isLoading={summaryQuery.isLoading} />

      <div className="grid grid-cols-3 gap-4 mt-6">
        <DeltaTrendChart data={trendQuery.data} loading={trendQuery.isLoading} />
        <SegmentOpportunityChart data={segmentQuery.data} loading={segmentQuery.isLoading} />
        <ChannelPerformanceChart data={channelQuery.data} loading={channelQuery.isLoading} />
      </div>

      <DecisionCardsTable data={cardsQuery.data} loading={cardsQuery.isLoading} />
    </div>
  );
}

```

- チャートは既存で使っているライブラリ（Recharts 等）をそのまま流用。
- SegmentOpportunityChart は BubbleChart / ScatterPlot を使用し、x=Incremental CLV, y=Segment Size, バブルサイズ=Δ¥, 色=BCG Quadrant。

3.5 フロント実行・テストコマンド

```

cd frontend

# ローカル開発
npm run dev

# 単体テスト（もしReact Testing Library等を使用していれば）
npm test -- DecisionConsolePage.test.tsx

```

4. Wolfram ONE / Wolfram Language での補完スクリプト

4.1 目的

- Python の実装と整合しているか **検算** する
- Segment Opportunity Map / BCG ラベルの閾値調整などを **対話的に探索** する
- 将来、Wolfram のノートブックをそのまま「高度分析モード」として顧客に見せることも可能にする

4.2 入力フォーマットの想定

- `policy_evaluations.csv`
 - 上記 `policy_evaluations` テーブルの抜粋
- `segments.csv`
 - 上記 `segments` テーブル

4.3 Global Summary の計算 (Wolfram)

```

(* Load data *)
policyData = Import["policy_evaluations.csv"];

(* 期間フィルタ (例: 2025-10-29~2025-11-25) *)
fromDate = DateObject[{2025, 10, 29}];
toDate   = DateObject[{2025, 11, 25}];

withinRangeQ[row_] := Between[
  DateObject[row["period_end"]],

```



```

    {fromDate, toDate}
];

filtered = Select[policyData, withinRangeQ];

deltaVals = filtered[All, "delta_yen"];
casVals   = filtered[All, "cas"];

totalDelta = Total[deltaVals];
avgDelta   = If[Length[deltaVals] > 0, Mean[deltaVals], 0.];

meanCAS    = If[Length[casVals] > 0, Mean[casVals], 0.];

(* CVaR 10% *)
sorted = Sort[deltaVals];
alpha = 0.1;
cutoff = Max[1, Floor[Length[sorted]*alpha]];
cvar = Mean[Take[sorted, cutoff]];

Print["Total ΔYen: ", totalDelta];
Print["Avg ΔYen / Policy: ", avgDelta];
Print["Mean CAS: ", meanCAS];
Print["CVaR 10%: ", cvar];

```

4.4 Segment Opportunity Map の生成

```

segments = Import["segments.csv"];

(* 分位数で High/Low を決める *)
clvVals   = segments[All, "incremental_clv"];
sizeVals  = segments[All, "user_count"];

clvThresh = Quantile[clvVals, 0.7];
sizeThresh = Quantile[sizeVals, 0.7];

bcgQuadrant[clv_, size_] :=
  Which[
    clv >= clvThresh && size >= sizeThresh, "Star",
    clv >= clvThresh && size <  sizeThresh, "Niche",
    clv <  clvThresh && size >= sizeThresh, "Volume",
    True,                                     "Low"
  ];

segmentsWithBCG = Map[
  Append[#, "bcg_quadrant" -> bcgQuadrant[#[ "incremental_clv"], #[ "user_count"]]] &,
  segments
];

(* バブルチャート: x=CLV, y=Size, ラベル=セグメント名 *)
BubbleChart[
  segmentsWithBCG[All, {"incremental_clv", "user_count", "total_delta_yen"}],
  ChartLegends -> Placed[{"Star", "Niche", "Volume", "Low"}, Above],
  Frame -> True,
  FrameLabel -> {"Incremental CLV", "Segment Size"},
  ChartLabels -> Placed[segmentsWithBCG[All, "name"], Center],
  PlotTheme -> "Business"
]

```

このスクリプトで決めた `bcg_quadrant` ロジックを、そのまま Python 側に移植すれば整合がとれます。

4.5 Channel Performance の検証

```

channelAgg = GroupBy[filtered, #[ "channel" ] &];

channelSummary = KeyValueMap[
  Function[{channel, rows},
    <|

```

```

    "channel" -> channel,
    "total_delta_yen" -> Total[rows[All, "delta_yen"]],
    "risk" -> StandardDeviation[rows[All, "delta_yen"]],
    "mean_cas" -> Mean[rows[All, "cas"]]
  ]>
],
channelAgg
];

ListPlot[
  channelSummary[All, {"risk", "total_delta_yen"}],
  PlotStyle -> PointSize[0.02],
  PlotRange -> All,
  Frame -> True,
  FrameLabel -> {"Risk", "ΔYen"},
  PlotLegends -> channelSummary[All, "channel"]
]

```

5. 実装ステップのまとめ (Claude向けTODO順)

- データモデル確認 & マイグレーション
 - 既存DBに `policy_evaluations`, `segments` に相当するテーブルがあるか確認し、不足分のカラム (`risk`, `cas`, `budget`, `reach_users` など) を追加する。
- バックエンド集計ロジックの実装
 - `backend/engine/models/console.py` に Pydantic モデルを追加。
 - `backend/engine/services/console.py` に5つの集計関数を実装。
 - `backend/gateway/routers/console.py` に FastAPI ルーターを追加し、`app.py` でマウント。
- ユニットテスト
 - テストデータを投入し、`/api/v1/console/*` が期待通りの JSON を返すか `pytest` で検証。
- フロントエンドの型 & API クライアント実装
 - `frontend/src/types/console.ts` に型定義追加。
 - `frontend/src/lib/consoleClient.ts` に `fetch` 関数を追加。
- DecisionConsolePage の UI 実装
 - 期間切り替えボタン (Last 14d / 28d / 56d)
 - KPI カード (Total Δ¥, Avg Δ¥, Mean CAS, CVaR)
 - 3チャート (Δ¥ Trend, Segment Opportunity Map, Channel Performance)
 - Decision Cards テーブル (フィルタ付き)
- i18n (日本語/英語切替) 対応
 - 既存の i18n 機構に合わせて、新規テキストを辞書ファイルへ登録。
- Wolfram ノートブックの作成 (任意だが推奨)
 - 上記 .wl スニペットをベースに、`CQOx_GlobalGrowthConsole_Validation.nb` のような notebook を1つ用意し、Python 実装との検算・デバッグに使えるようにする。

What this does (この仕様が実現すること)

- ゴール
 - CQOxに「世界レベルの意思決定コンソール」ページを追加し、直近14/28/56日分のマーケティング施策を **Δ¥・リスク・CAS・セグメント・チャネル** の5軸で一枚にまとめて可視化できるようにする。
- 入力
 - 既存の因果推論結果テーブル (`policy_evaluations` など)、セグメントメタデータ (`segments`)、期間やチャネル・セグメントのフィルタ。
- 出力・副作用
 - `/console` ページで、
 - KPIカード (Total Δ¥ / Avg Δ¥ / Mean CAS / CVaR)
 - Δ¥トレンド、Segment Opportunity Map、Channel Performance
 - Decision Cards テーブル
 を表示し、CMOが週次レビューを1ページで完了できる状態を作る。

- バックエンドでは、新しい集計API群 `/api/v1/console/*` を提供し、将来 MMM や他モジュールにも再利用可能な統計ビューを構築する。
 - **実行・検証**
 - Python側は `pytest` で API の数値整合性をテストし、
 - Wolfram ONE 側のスクリプトで独立に同じメトリクスを計算して結果を突き合わせることで、実装ミスや境界条件のバグを早期に検出できる。
-

Expert insight (Google/Meta/NASAレベル)

トップレベルのダッシュボードは、「全指標を載せる」ためではなく、「意思決定を固定フォーマット化する」ために存在する。

Δ¥・リスク・CAS・ポートフォリオ・セグメント・チャネルという軸を、毎週まったく同じ並びで見せ続けることで、「今週も同じルールで判断した」という **ガバナンスと説明責任** が生まれます。世界トップ企業の“Command Center”は、派手な可視化ではなく、**同じフォーマットで意思決定を積み重ねる**ことを最優先に設計されています。